

Distributed Caching

 systemdesignschool.io/fundamentals/distributed-caching

Modulo Remapping: Adding 1 Node Disrupts Most Keys

	hash	% 4	% 5	Result
key-A	5	Node 1	Node 0	remapped
key-B	12	Node 0	Node 2	remapped
key-C	7	Node 3	Node 2	remapped
key-D	8	Node 0	Node 3	remapped
key-E	14	Node 2	Node 4	remapped
key-F	9	Node 1	Node 4	remapped
key-G	3	Node 3	Node 3	same node
key-H	20	Node 0	Node 0	same node

6 of 8 keys remapped (75%) — each becomes a cache miss on the new node

A single Redis node with 64 GB of RAM serves well for small-to-medium workloads. But when the working set exceeds one machine's memory or throughput capacity, you need multiple cache nodes. The question becomes: which node holds which keys?

Why Shard the Cache

Two forces push toward multiple nodes:

Memory limit. A single machine has finite RAM. If the working set is 200 GB, no single node can hold it. Spreading keys across 4 nodes gives 50 GB each.

Throughput limit. A single Redis node handles roughly 100,000–200,000 operations per second (depending on value size, command type, and hardware). If the application's workload exceeds what a single node can sustain, sharding becomes necessary regardless of memory capacity.

Sharding splits the key space across multiple nodes. Each node is responsible for a subset of keys. The client determines which node to contact for a given key, routes the request, and gets a response. From the application's perspective, the distributed cache behaves like one large cache.

Hash-Based Routing

The simplest approach: hash the key and use modulo to pick a node.

```
node = hash(key) % num_nodes
```

Modulo Remapping: Adding 1 Node Disrupts Most Keys

	hash	% 4	% 5	Result
key-A	5	Node 1	Node 0	remapped
key-B	12	Node 0	Node 2	remapped
key-C	7	Node 3	Node 2	remapped
key-D	8	Node 0	Node 3	remapped
key-E	14	Node 2	Node 4	remapped
key-F	9	Node 1	Node 4	remapped
key-G	3	Node 3	Node 3	same node
key-H	20	Node 0	Node 0	same node

6 of 8 keys remapped (75%) — each becomes a cache miss on the new node

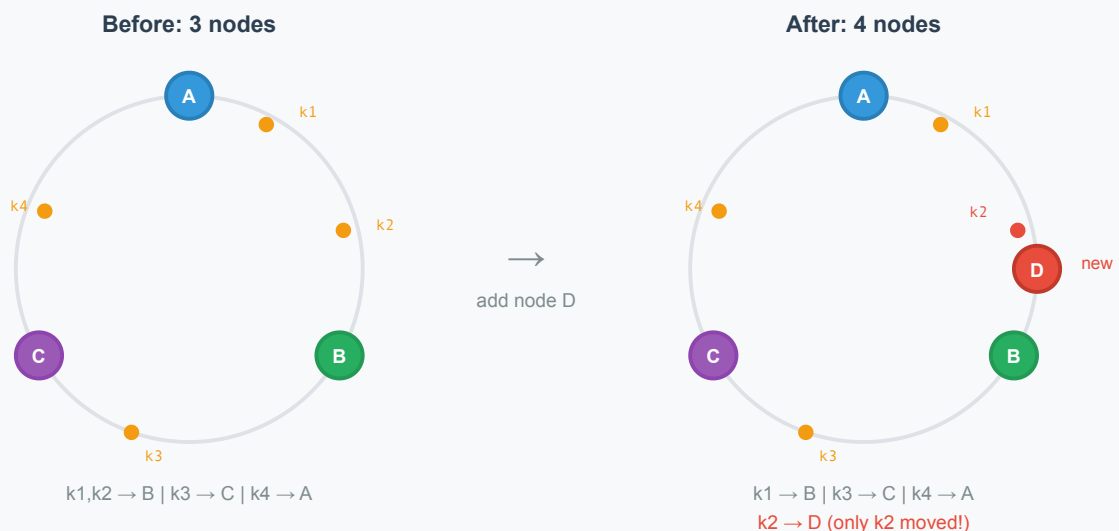


This distributes keys evenly. But adding or removing a node changes `num_nodes`, which changes the modulo result for nearly every key. A 4-node cluster expanding to 5 nodes remaps ~80% of keys. Those remapped keys become misses on the new node—the database absorbs a burst of traffic until the cache re-warms.

Consistent Hashing

Consistent hashing solves the remapping problem. When a node is added or removed, only the keys previously assigned to the affected node are remapped—roughly $1/N$ of the total keys, where N is the number of nodes. The other $(N-1)/N$ keys stay on their current nodes.

Consistent Hashing: Adding a Node



The concept: nodes and keys both map onto a circular hash space (a ring). Each key is assigned to the first node encountered when walking clockwise from the key's position on the ring. Adding a new node between two existing nodes only takes keys from its clockwise neighbor—all other assignments are unchanged.

The [Consistent Hashing](#) article in the Data Partitioning section covers the algorithm in detail, including virtual nodes for load balancing. The same technique applies directly to cache sharding.

Redis Cluster: Hash Slots

Redis Cluster uses a fixed set of 16,384 **hash slots** instead of a continuous ring. Each key maps to a slot via $\text{CRC16}(\text{key}) \% 16384$. Each node owns a range of slots.

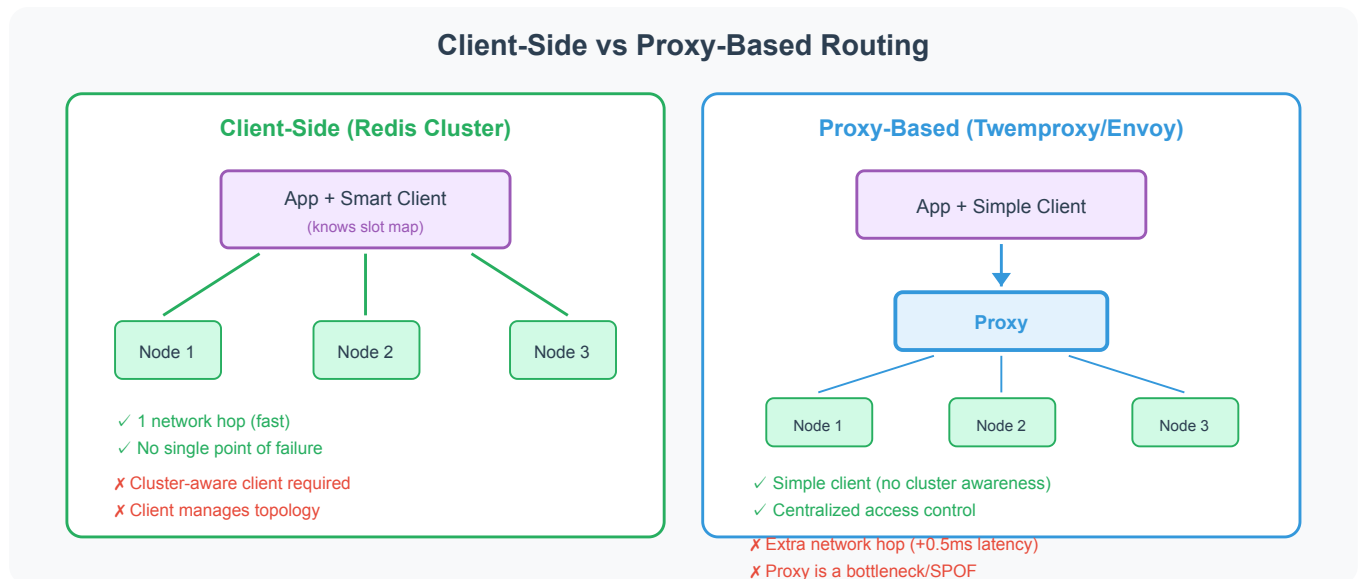
When scaling from 3 to 4 nodes, you (or your managed service) reshard some slots from existing nodes to the new one. Keys in moved slots are migrated—clients may see brief redirections during the migration, but other slots are unaffected.

Nodes	Slots per Node	Key Remapping on Add
3 → 4	5,461 → 4,096	~25% of keys
4 → 5	4,096 → 3,277	~20% of keys
6 → 7	2,731 → 2,341	~14% of keys

Redis Cluster provides tooling to assign and rebalance slots; managed services often automate this. The client library discovers the slot map from any node and routes requests directly to the correct node without a proxy.

Hash tags allow related keys to land on the same node. The key `{user:123}:profile` and `{user:123}:settings` both hash on `user:123`, placing them on the same slot. This enables multi-key operations (like `MGET`) that require co-located keys.

Client-Side vs Proxy-Based Routing



Two architectures for routing requests to the correct shard:

Client-side routing. The application client library knows the shard map and routes requests directly. Redis Cluster uses this approach. The client is "smart"—it maintains a local copy of the slot-to-node mapping and sends each command to the correct node. This avoids an extra network hop but requires cluster-aware client libraries.

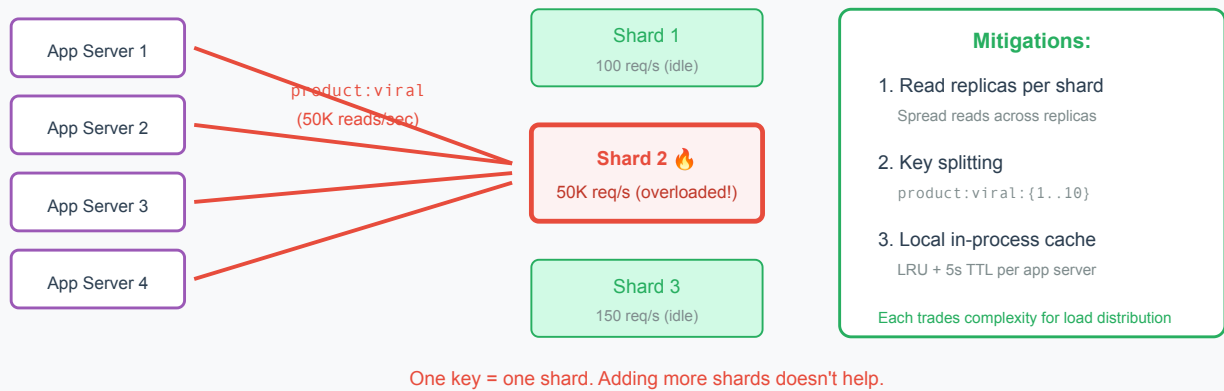
Proxy-based routing. A proxy sits between the application and the cache nodes. The application sends all requests to the proxy, which routes them to the correct shard. The application uses a simple, non-cluster-aware client. The proxy adds one network hop of latency but simplifies the client. Tools like [Twemproxy](#) and [Envoy](#) support this pattern.

In Redis Cluster deployments, client-side routing is typical because the latency savings of direct connections outweigh the slight client complexity. Proxy-based routing is useful when the application cannot use a cluster-aware client or when you need centralized access control.

Hot Key Problem

Sharding distributes keys evenly by hash value, not by access frequency. If one key receives 50% of all reads (a viral product page, a breaking news article), its shard handles 50% of total load while other shards idle. The hot shard becomes a bottleneck regardless of how many total nodes exist.

Hot Key Problem: Sharding Doesn't Help



Mitigations for hot keys:

Read replicas per shard. Each shard has one or more read replicas. Hot-key reads spread across the primary and its replicas. This helps but adds complexity and memory cost.

Key splitting. Append a random suffix to the hot key (e.g., `product:viral:{1..10}`). The client reads from a random variant. Ten variants can spread the load across up to 10 shards, depending on how slots are distributed. The write path must update all variants, adding write amplification.

Local caching. Cache the hot key in application-level memory (an in-process LRU cache with short TTL). This reduces Redis traffic for hot keys significantly, depending on the local cache's hit rate and TTL. This reduces Redis load but introduces a second consistency layer.

From there, the focus shifts to keeping the distributed cache available when individual nodes fail—replication, failover, and the tradeoffs involved.